Lead Intel: Project Guide

Oct 18, 2026 · @LASSILOVER

1. What Lead Intel is

Lead Intel is a monthly-subscription website that works as an AI growth partner for Indian local businesses: gyms, clinics, salons, car-detailing studios, restaurants and similar shops. It catches every customer enquiry, tells the owner which ones matter most, helps them post on social media, explains each week what to do next, and looks after their online reviews.

The problem it solves

A local business gets enquiries from many places at once: WhatsApp, Instagram, Facebook, Google, Justdial, Sulekha, Practo, its own website, walk-ins. They land in different apps, nobody tracks them, and good customers are lost because no one replied in time. On top of that, the owner knows they should post regularly and answer reviews, but has no time or marketing skill.

Who it is for

- **The business owner**: sees every enquiry in one inbox, gets an alert for hot leads, approves AI-written posts and review replies.
- **Their staff**: log in with their own account and handle enquiries, but cannot change settings or billing.
- **You (the platform owner, "admin")**: see every business, payments, emails and the go-live checklist.

The four main features

Feature	What it does for the owner
Lead manager	Every enquiry from 36 platforms in one inbox. The AI reads each message, scores it, and flags hot leads so the owner calls them first.
AI social media growth	The AI plans a 7-day posting calendar (including Indian festivals), writes captions, hashtags and reel scripts in the brand's voice, and publishes on schedule after the owner approves.

Feature	What it does for the owner
Weekly growth review	Every Monday the AI looks at the week's numbers and sends a short report with the top 3 actions to take.
Reputation	New Google reviews arrive automatically, the AI drafts a reply, and happy customers get a polite "please review us" WhatsApp.

A **live feed** ties it together: new leads, scores and hot-lead alerts appear on the dashboard the moment they happen.

Two rules hold everywhere: plain code calculates the numbers and the AI only explains them, and **nothing is posted or sent without the owner's approval**.

How it earns money

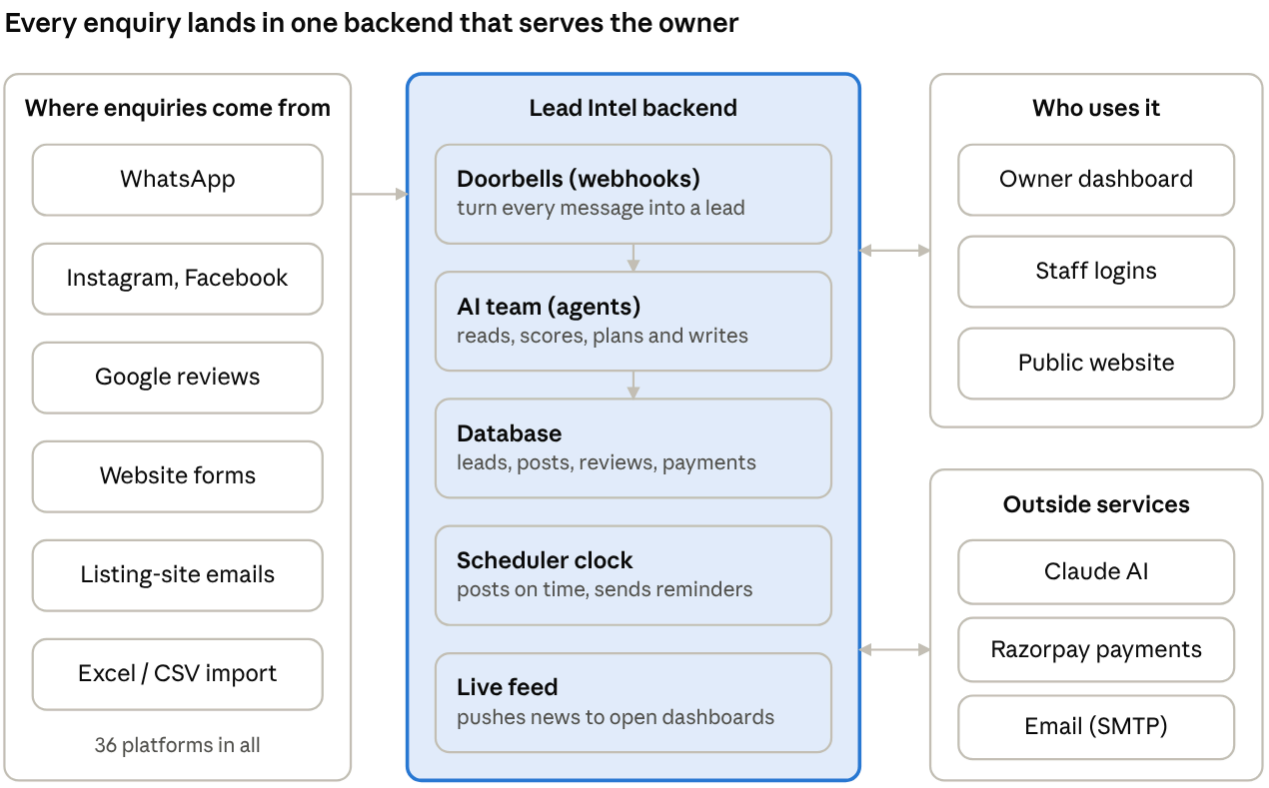
Every new business starts with a 14-day free trial, then picks a monthly plan paid through Razorpay.

Plan	Price per month	Main limits
Starter	₹999	Lead inbox and AI scoring, AI posts, review replies · 1 staff login, 2 social accounts, 1 GB media
Growth	₹2,499	Everything in Starter + weekly growth review and automatic review requests · 3 staff, 6 accounts, 5 GB
Pro	₹4,999	Everything in Growth · 10 staff, 20 accounts, 20 GB, priority support

When a trial or plan ends, the dashboard asks the owner to choose a plan, but incoming enquiries are still saved so no customer is lost. Prices and limits live in one file (`backend/app/plans.py`) and can be changed any time.

2. How it works: the big picture

Every enquiry, from any platform, lands in one backend. The backend saves it, lets the AI team read and score it, and shows the result to the owner straight away.



Lead Intel architecture · sources, backend, users, outside services

Read it left to right. Enquiries ring a doorbell, the AI team fills in and scores the lead, and the database keeps it. The scheduler clock posts on time and sends reminders, and the live feed pushes news to the owner's dashboard. The backend calls three outside services: Claude for AI, Razorpay for payments and an email provider. On the live site the backend and its database run on Railway.

3. How the work was divided

The backend was built in 26 tasks, grouped into three stages. Each stage stands on the one before it: first the lead manager (the core), then the growth features, then everything a real paid product needs to go live.

Stage	Tasks	Goal	Result
A. Foundations and lead manager	1–11	Catch every enquiry, store it safely, let the AI read and score it, and count the results	A working lead inbox with an AI team and analytics

Stage	Tasks	Goal	Result
B. Growth features	12–18	Social media posting, the weekly growth review, reviews and reputation, and the live feed	The full "AI growth partner"
C. The business side	19–26	Logins, payments, real platform connections, uploads, emails, more lead sources, a public website with legal pages, and a production-ready server	A product that can take real customers and real money

The React dashboard (the screens owners click) is being built separately by a friend from `FRONTEND_PROMPT.md`, which describes every screen and every backend address it calls.

The working rules we followed

1. **One file at a time.** Each file was written, explained in plain words, and checked before moving to the next.
2. **Mock mode first.** Every outside service (the AI, Razorpay, WhatsApp, Instagram, Google, email) has a pretend version that behaves like the real one but costs nothing. The whole product runs free on a laptop; real keys are added later in `.env` and nothing else changes.
3. **Test everything.** Every task ended with automatic checks. The project now has **418 checks**, and all of them pass on both SQLite (the laptop database) and PostgreSQL (the live database).
4. **A "Try it" at the end of each task.** Short steps to see the new feature working at `http://127.0.0.1:8000/docs`.
5. **Safety by default.** Each business sees only its own data, secrets are encrypted, and the live server refuses to start with unsafe settings.
6. **Honest wording.** The public website promises only what the product really does today.

4. Stage A: foundations and the lead manager (Tasks 1–11)

Stage A built the heart of the product: an enquiry arrives from any platform, is saved safely, is read and scored by a small AI team, and shows up in the numbers.

Task 1: Project setup

We created the project folders, the Python environment and the FastAPI server, plus a `/health` address that answers "I'm alive". All settings (keys, addresses, mock switches) live in one `.env` file, read by `config.py`, so nothing secret is written into the code.

Task 2: The database

We designed the tables that hold everything: businesses, leads (one per customer enquiry), messages (each chat line), lead analysis (what the AI concluded) and agent runs (a log of every AI job). `models.py` describes the tables and `database.py` opens the connection. On the laptop this is a single SQLite file, `data/leads.db`.

Task 3: The data rules

`schemas.py` checks every piece of data that comes in, before it is saved: a phone number must look like a phone number, a lead status must be one of the allowed steps (new, contacted, quoted, booked, won, lost), and so on. Bad data gets a clear error (code 422) and never reaches the database.

Task 4: Demo data

`seed/seed_data.py` fills the database with 3 realistic demo businesses: Iron Pulse Fitness (gym), Gloss Garage Detailing and SmileCare Dental Clinic. Together they have about 300 leads, 170 posts and 60 reviews, so every feature can be shown and tested with believable data.

Task 5: The lead API

The addresses the dashboard calls to work with leads: list and filter leads, open one lead with its full chat, add a message, change its status, and ask the AI to analyse it again. Businesses have their own addresses too (`/businesses`).

Task 6: Webhooks for any platform (the "doorbells")

A webhook is an address other services ring when something happens, for example WhatsApp calling us when a customer sends a message. We built doorbells for WhatsApp, Instagram and the business's website, plus one **universal** doorbell for any of 36 platforms. A translator (`normalizer.py`) turns each platform's different format into one common shape, so everything after it only understands one format. The WhatsApp and Instagram formats copy Meta's real ones, so going live later changes only the doorbell.

Task 7: The AI connector

`agents/llm.py` is the one place that talks to the AI model (Claude). It gives three things: **mock mode** (free rule-based answers, so we could build without paying), **structured answers** (the AI fills in a form instead of writing a chatty paragraph), and a **safety net** (if the real AI fails, it falls back to the mock answer, so a lead is never lost).

Task 8: The Intake and Tone agents

The **Intake agent** is the receptionist. From "Hi I'm Rahul from Dwarka, need ceramic coating for my Creta this Saturday" it fills the lead card: name Rahul, area Dwarka, service ceramic coating, car Hyundai Creta, day Saturday. It knows what matters in each industry (batch timing for a gym, car model for a garage). The **Tone agent** reads how the customer feels: urgent, frustrated or casual. Frustrated wins over urgent, because an annoyed customer is about to go to a competitor.

Task 9: The Scoring agent

Each lead gets a score from 1 to 100 for "how likely is this person to buy?". It is a deliberate mix: **fixed points** from plain code (source, phone shared, number of messages, tone) plus an **AI adjustment** of −20 to +20 for buying intent ("Book me for Saturday 10am" scores higher than "just checking prices"). Every point is listed in a breakdown, so the owner can see exactly why a lead got its score.

Task 10: The lead workflow

Using LangGraph, the agents are joined into one assembly line that runs in the background for every new message: Intake → Tone → Scoring → Priority → hot alert or normal queue → save. The **Priority router** uses plain rules: a lead is hot if its score is 70+, or it is frustrated with 40+, or urgent with 65+. These numbers were tuned so about 1 in 5 leads is hot; if everything were hot, the owner would stop trusting the alert.

Task 11: Analytics

`analytics_service.py` is the calculator, with no AI involved, so no number can be invented. For any period it answers: how many enquiries, sales, ₹ revenue and reply speed; which platform brings enquiries and which brings money; the funnel (contacted → quoted → booked → won); busiest hours and days; which services sell; and the week-by-week trend.

5. Stage B: the growth features (Tasks 12–18)

Stage B turned the lead inbox into a growth partner: the AI now plans and writes posts, tracks which posts bring money, reviews the week, and looks after reviews, all shown live on the dashboard.

Task 12: Social accounts and posts

Each business can list its Instagram, Facebook and Google accounts and keep its posts in the app. A post moves through clear steps: idea → draft → scheduled → published (or failed). Most importantly, we built the **post → lead → money link**: every caption ends with a WhatsApp link carrying a small code such as "(code P12)", so when a customer taps it, the lead is tied to post #12 and the owner can see which post earned real ₹.

Task 13: The Content Planner

The Content Planner agent plans the next 7 days of posts. It looks at what has worked for this business before, which services earn money, which platforms it uses, posts already planned, and an **Indian festival calendar** (`festivals.py`) with an angle for each industry, for example Diwali + car detailing = "pre-Diwali shine". Each planned post comes with the reason behind it, tied to the numbers, so the owner can trust it.

Task 14: The Writer

The Writer agent turns each idea into a ready post: a caption with a scroll-stopping first line and a clear call to action, the right hashtags for the platform, a scene-by-scene reel script or slide-by-slide carousel, and a shot list telling the owner exactly what photo or video to take. It writes in the business's own voice: energetic Hinglish for a gym, premium for a detailing studio, caring for a clinic.

Task 15: The scheduler and performance tracker

An alarm clock inside the server wakes every minute. It publishes posts whose time has come, refreshes each post's numbers (reach, likes and so on), sends due "please review us" messages, and starts the Monday growth reviews. The owner approves a post, picks a time, and the scheduler does the rest. In this stage publishing was mock (pretend); real publishing came in Task 21.

Task 16: The weekly growth review

A second LangGraph workflow runs every Monday at 9am, or on demand:

1. **Facts**: plain code compares this week with last week and adds the 90-day picture.

2. **Growth Analyst** (AI) and **Presence Auditor** (plain rules) work side by side. The analyst picks the 3–6 findings that matter, such as "Instagram brings enquiries but not customers". The auditor ticks a checklist: posting enough, every enquiry answered, listed on the industry's own apps, good Google rating.
3. **Strategist** (AI) picks the 3 actions that bring the most customers for the least effort.
4. **Report Writer** (AI) turns it all into one short message the owner can read on WhatsApp in 30 seconds.

Task 17: Reputation

Reviews are saved without duplicates and matched to the customer. The **Review Reply agent** tags each review (positive, neutral or negative; topics like staff, price, waiting time) and drafts a reply. Replies are drafts: the owner approves first, and replies to negative reviews are never posted automatically. When a lead is marked won, a polite "please review us" WhatsApp is queued and sent only between 10am and 8pm. A summary shows the rating, reply rate, common praise and complaints.

Task 18: The live feed and the README

The **event bus** is a radio station inside the server: when something happens (new lead, hot lead, post published, unhappy review) it broadcasts to every open dashboard of that business, using a technique called SSE. Each business has its own channel. If a phone loses signal for a moment, it reconnects and gets the events it missed. We also wrote the README: setup on Windows, demo logins, the AI team, the address map and the folder map.

6. Stage C: the business side (Tasks 19–26)

Stage C added everything a real paid product needs: accounts, payments, real platform connections, uploads, emails, more lead sources, a public website with legal pages, and a server that is safe to run live.

Task 19: Login and accounts

Owners sign up, log in and log out, change their password and invite staff. There are three roles: **owner** (everything), **staff** (handles leads, cannot change settings or billing) and **admin** (you, sees every business). Passwords are stored scrambled (scrypt), never as plain text. Every request checks who is logged in and whether the data belongs to their business; asking for another business's data gets "not found", so it cannot even be guessed that it exists.

Task 20: Subscriptions and payments

The 14-day free trial, the three plans (Starter, Growth, Pro) and payments through **Razorpay**, including monthly renewals and cancellations. Every payment message from Razorpay is checked with a secret signature, so nobody can fake a "paid". When a trial or plan ends, the dashboard replies "choose a plan" (code 402), but new enquiries are still saved. Plan limits (staff seats, social accounts, storage) are enforced, and the admin gets a money page.

Task 21: Connecting the real platforms

Owners click **Connect** for WhatsApp (Meta's Embedded Signup), Instagram, a Facebook Page and Google Business Profile. Their access keys are stored **encrypted**. One Meta doorbell (`/webhooks/meta`) receives messages for all businesses and routes each to the right one. Replies, posts and review replies now go out for real. The app also respects Meta's **24-hour rule**: a business can reply freely only within 24 hours of the customer's last message. Google reviews are fetched every 30 minutes.

Task 22: Photo and video uploads

A media library per business. Each file is checked by its actual content, not just its name. Photos are cleaned: converted to JPEG, turned upright, hidden location data removed, and an Instagram-ready 4:5 crop made. Storage space follows the plan (1, 5 or 20 GB). Instagram needs a public link to a photo to publish it, so files are served from a public `/files/...` address.

Task 23: Forgot password and emails

Password reset by email, plus welcome emails, staff invites, "trial ends in 3 days" and "trial ended" reminders, payment receipts and "payment failed" notices. Emails wait in a queue, are never sent twice, and are retried if the email provider is down. They go through any SMTP provider (Brevo has a free plan). In mock mode they are saved as files you can open in a browser. The admin has an email log.

Task 24: More ways for enquiries to arrive

Lead emails: Justdial, Sulekha, Practo, IndiaMART and similar sites send enquiries by email. Each business gets its own private email address; the owner tells Gmail to forward those emails there, and each one becomes a lead with the name, phone, requirement and area read out of it. The reader skips helpline numbers, handles forwarded emails, and even picks up Gmail's forwarding confirmation code. **Excel/CSV import:** an owner can

upload their old customer register. A preview shows what will be saved; column names, ₹ amounts, Indian dates and statuses are understood, and customers already in the system are skipped.

Task 25: The public website and legal pages

A home page, pricing, about and contact page, plus the legal pages Razorpay requires: Terms, Privacy (written for India's data-protection law, the DPDP Act 2023), Refunds, Shipping and Data deletion. Owners can download all their data as a zip file or delete their account. A go-live checklist (`/admin/launch-check`) lists what is still missing before launch. The legal pages are a careful starting template; a lawyer should review them before launch.

Task 26: Production-ready

The server is now ready for the real world: it runs on **PostgreSQL** (all 418 checks pass on it), database changes are applied safely through numbered **migrations**, a nightly **backup** is written and its restore was tested, and every reply carries security headers. The live server **refuses to start** with a demo secret key, without https, or with demo accounts in the database. A `Dockerfile` and `DEPLOY.md` describe exactly how to put it on **Railway**. The last step, actually going live, is waiting on two things from you (see section 9).

7. The technology, in plain words

Every piece below is widely used and well documented, so another developer can pick the project up easily. The code tools are free; the AI, Razorpay, Railway and email provider charge only once used for real.

Piece	What it is	Why we chose it
Python 3.12	The programming language of the backend	Easy to read, and the best language for AI work
FastAPI	The framework that answers web requests	Fast, simple, and builds the `/docs` test page automatically
SQLAlchemy	Talks to the database from Python	The same code works on SQLite and PostgreSQL
SQLite	A database stored in one file	Zero setup on the laptop
PostgreSQL	A full database server	Reliable for many businesses at once on the live site

Piece	What it is	Why we chose it
Alembic	Migrations: numbered database changes	Changes tables safely without losing data
Pydantic	Checks data against rules	Bad data is stopped before it is saved
Claude (Anthropic)	The AI model behind the agents	Strong at reading messages and writing in a brand voice
LangGraph	Joins agents into a workflow	Clear steps, can run agents side by side
Razorpay	Indian payment gateway	UPI, cards and monthly subscriptions in ₹
Meta and Google APIs	WhatsApp, Instagram, Facebook, Google reviews	Where local customers actually are
SMTP (e.g. Brevo)	Sends emails	Works with any email provider
Pillow	Edits images	Cleans and crops uploaded photos
Jinja2	Fills HTML templates	Builds the public website pages
pytest	Runs the automatic checks	418 checks guard against breaking things
Docker and Railway	Packages and hosts the server	Simple deploy from GitHub, about $5/month to start
React (by your friend)	The dashboard screens	Built separately from `FRONTEND_PROMPT.md`

8. Where things are, and how to run it

The project lives in `C:\Users\soumy\Desktop\lead agent` and has three top-level parts: `backend/` (everything built so far), `deploy/` (the email helper for Cloudflare) and the guides (`README.md` , `DEPLOY.md` , `FRONTEND_PROMPT.md`).

Folder map

Folder or file	What lives there
`backend/app/main.py`	Starts everything: the addresses, the scheduler clock, the live feed

Folder or file	What lives there
`backend/app/config.py`	All settings, read from `.env`
`backend/app/models.py` / `schemas.py`	The database tables / the data rules
`backend/app/plans.py`	The price list and what each plan includes
`backend/app/platforms.py` / `festivals.py`	The 36 platforms / the Indian festival calendar
`backend/app/auth/`	Passwords, the login guard, the paywall, the encryption vault
`backend/app/routers/`	The addresses, one file per area (leads, billing, media…)
`backend/app/services/`	The actual work: leads, analytics, posts, scheduler, billing, emails…
`backend/app/agents/`	The AI connector, the two workflows, one file per agent in `nodes/`
`backend/app/integrations/`	The real calls to Meta, Instagram and Google
`backend/app/site/`	The public website: pages, templates, styles
`backend/seed/`	Demo data, the traffic simulator, create admin, restore backup
`backend/migrations/`	The numbered database changes
`backend/tests/`	The 418 automatic checks
`backend/Dockerfile`	How Railway builds the live server

How to run it on your laptop (Windows)

1. Once: open the folder in VS Code, then in a terminal run `cd backend`, `python -m venv .venv`, `.\.venv\Scripts\python.exe -m pip install -r requirements.txt` and `.\.venv\Scripts\python.exe -m seed.seed_data --reset --analyze`.
2. Every time: `cd backend` then `.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload`.
3. Open `http://127.0.0.1:8000/docs` to try every feature, and `http://127.0.0.1:8000/` for the public website.
4. Log in with a demo account, for example `gym@demo.in` / `demo1234` (owner) or `admin@demo.in` / `admin1234` (admin).

5. To watch it live, run `.\.venv\Scripts\python.exe -m seed.simulate_traffic --every 3` in a second terminal: fake customers start messaging the demo businesses.
6. To run the checks: `.\.venv\Scripts\python.exe -m pytest tests -q`.

The demo passwords work only on the laptop; the live server refuses to start if demo accounts exist.

9. What is left before launch, and ideas for later

The backend is complete and tested; going live now depends mostly on accounts and approvals outside the code.

Before launch

- [ ] Create an empty **private** GitHub repository named `lead-agent` (github.com/new, no README), so the code can be pushed there
- [ ] Approve Railway's Hobby plan (about $5/month including $5 of usage), then the server and database go live on a new Railway project
- [ ] Add your own admin login in Railway's dashboard (`BOOTSTRAP_ADMIN_EMAIL` / `BOOTSTRAP_ADMIN_PASSWORD`), typed by you, then delete both variables
- [ ] Buy a domain (for example a `.in` name, about ₹500–1,000/year) and point it at Railway
- [ ] Open real accounts and add their keys: Razorpay (live keys), an email provider such as Brevo, a Meta app (WhatsApp, Instagram, Facebook), and Google Business Profile API access
- [ ] Allow 2–4 weeks for Meta's business verification and App Review before other businesses can connect
- [ ] Turn on the real AI (`MOCK_AI=false` plus an Anthropic key)
- [ ] Your friend finishes and deploys the React dashboard; then set `FRONTEND_ORIGIN`
- [ ] Fill in your company details for the legal pages, and have a lawyer review Terms, Privacy and Refunds
- [ ] Set up Cloudflare Email Routing for the private lead-email addresses
- [ ] Forward one real lead email from each platform you use (Justdial, Sulekha…) so the reader can be tuned to its exact layout
- [ ] Before May 2027 (DPDP Rules): automatically email, then delete, businesses inactive for 12+ months

Ideas for later

- Split the scheduler and live feed into their own service once there are many hundreds of businesses (today exactly one server copy runs them)

- More languages for posts and replies beyond English and Hinglish
- A mobile app or WhatsApp-only mode for owners who rarely open a laptop

10. Glossary

Word	Meaning
Agent	A small AI worker with one job, such as reading a message or writing a caption
API	The set of addresses a program calls to ask the backend for something
Backend	The part that runs on the server: data, rules, AI, payments. Users never see it directly
Frontend / dashboard	The screens the owner clicks, built in React
Endpoint (address)	One specific API address, such as `/leads`
Webhook (doorbell)	An address another service calls when something happens, such as a new WhatsApp message
Lead	One customer enquiry, with its messages, score and status
Hot lead	A lead the rules say to call now (high score, or frustrated, or urgent)
Mock mode	A free pretend version of an outside service, used for building and testing
Database / table	Where data is stored, organised like spreadsheets
Migration	A numbered, safe change to the database tables
Workflow (LangGraph)	Several agents joined in a fixed order, like an assembly line
SSE (live feed)	A way for the server to push live updates to an open browser
Encryption	Locking data so only our server, with its secret key, can read it
Token	The temporary pass a user gets after logging in
402 / 404 / 422	Reply codes: "choose a plan" / "not found" / "data broke a rule"

Word	Meaning
Paywall	The check that blocks the dashboard when the trial or plan has ended
SMTP	The standard way programs send email
OAuth (Connect button)	The safe way an owner lets us use their Instagram or Google without sharing a password
24-hour rule	Meta's rule that a business may reply freely only within 24 hours of the customer's last message
DPDP Act	India's Digital Personal Data Protection Act, 2023: the law on handling personal data
Docker / Dockerfile	A recipe that packages the server so it runs the same everywhere
Railway	The hosting company that will run the live server and database
Environment variables / `.env`	Settings and secret keys kept outside the code